# FPS-Voronoi: From Mock 2-D to Real LiDAR

Adaptive point-cloud sampling — and measuring how much consecutive frames differ

---

Discussion deck · full detail in README.md

CONTEXT

# The pipeline in one slide

- Goal: pick a small sample set S that represents a large point cloud P well.
- Each cloud point belongs to its nearest sample → implicit Voronoi cells.
- Phase 1 — five geometric primitives (membership, covering radius, occupancy, nearest-pair, Delaunay neighbours).
- Phase 2 — one fused pass + three detectors:
  - Coverage gap (cell too large) · Separation (samples too close) · Vanishing (empty cell).
- Phase 3 — correction loop: insert / evict with cheap one-hop updates, under an edit budget.
- Iterated, it turns raw FPS into an adaptive resampler.

AGENDA

# What we did today

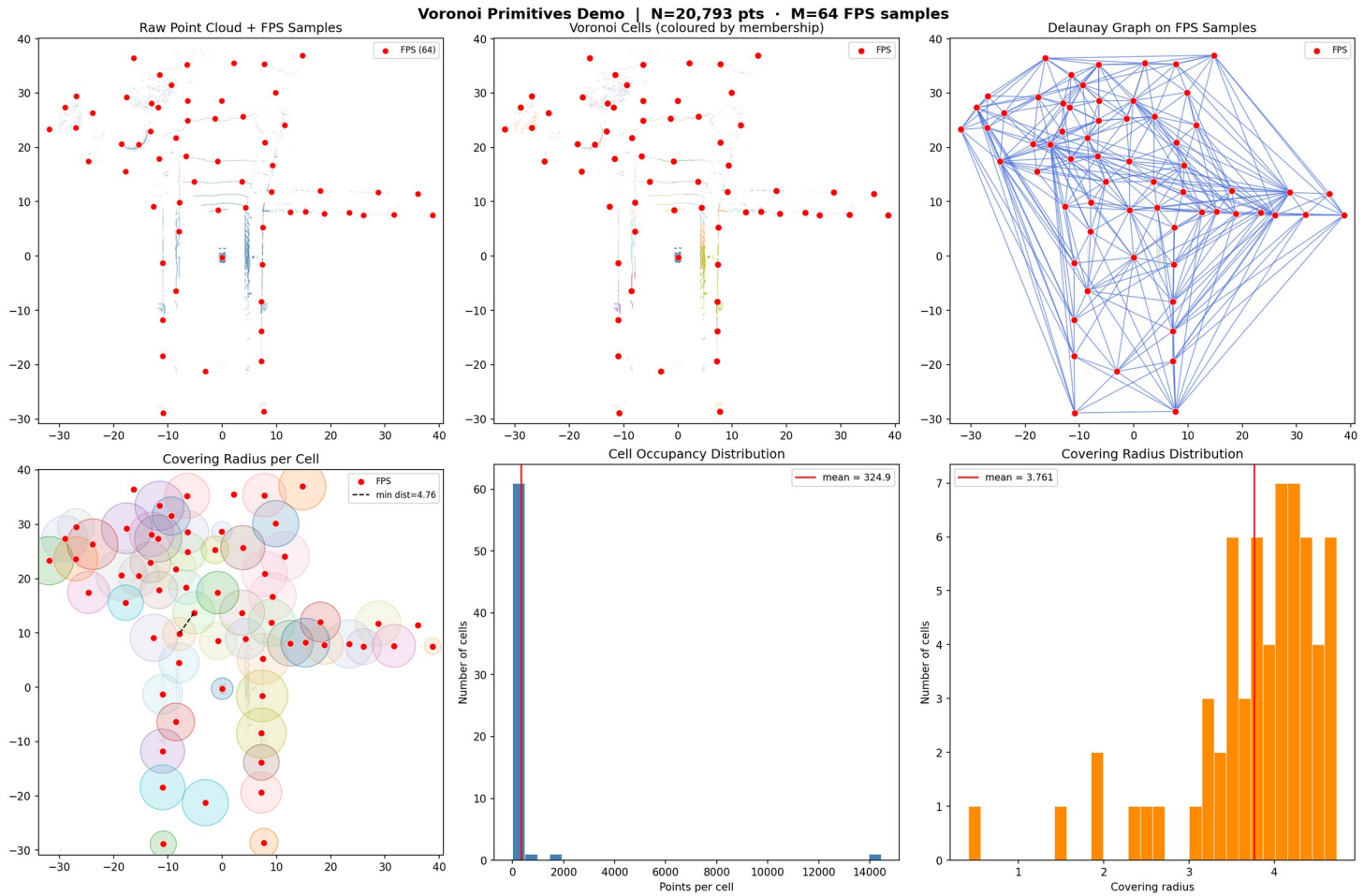
- 1. Moved the whole pipeline from mock 2-D data onto real 3-D LiDAR (KITTI + nuScenes).
- 2. Built a temporal loop: carry the sampling across consecutive frames and measure how much it must adapt.
- 3. Added a Chamfer quality metric — and used it to ask whether reusing the previous frame's samples is as good as rebuilding.

- Theme for discussion: a cheap, reused sampling vs. a from-scratch one — what we gain, what we pay, and why.

# Real LiDAR: the engine was already dimension-agnostic

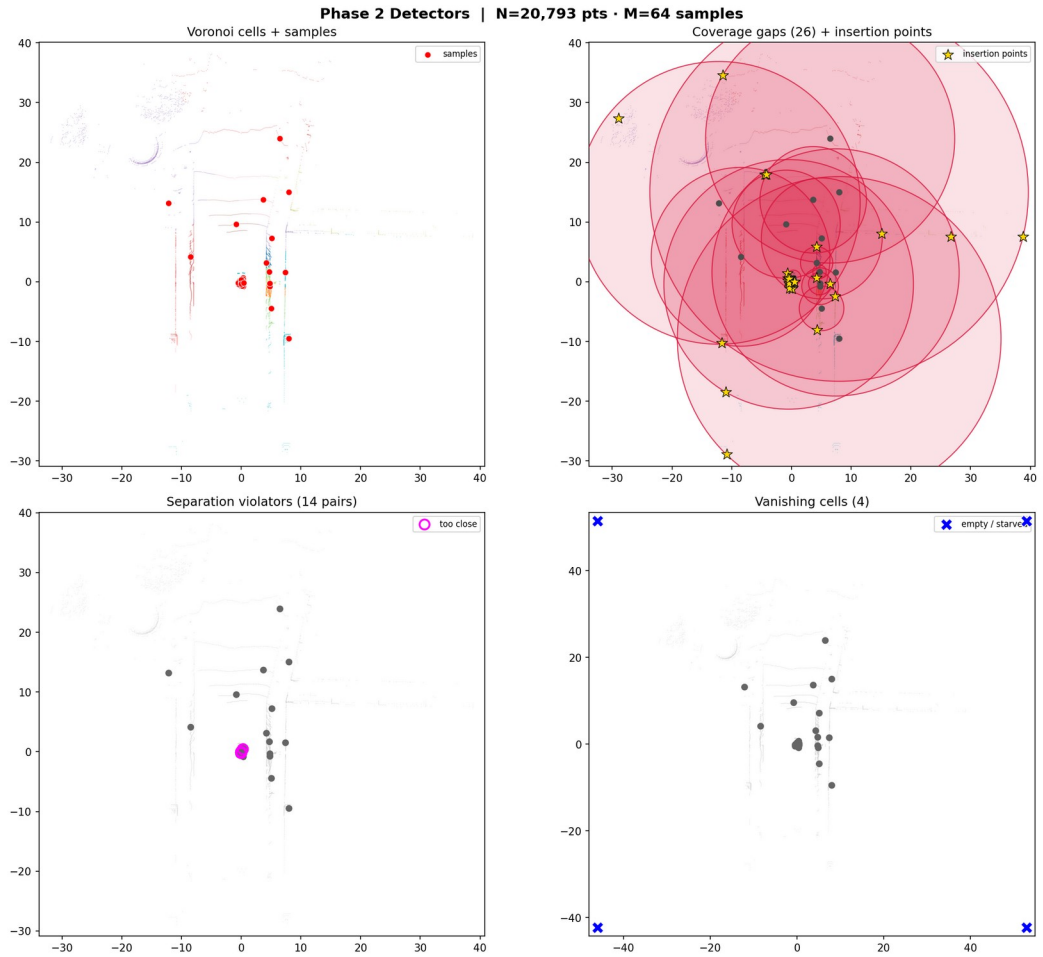
- Every primitive uses distance only (cdist / norm / Delaunay) → works in 2-D or 3-D unchanged.
- So real data is a loading concern, not an algorithm change. One shared loader (data_io.py):
  - Auto-detects format: KITTI .bin (x,y,z,intensity) · nuScenes .pcd.bin (x,y,z,intensity,ring).
  - dims = 3 (full) or 2 (top-down, keeps the original visualizations).
  - Optional max_range crop and min_z ground removal.
- Why crop / remove ground: on raw LiDAR, FPS grabs sparse far/high outliers and leaves one giant central cell (~22k of 35k points).

# Primitives on a real nuScenes frame (3-D, cropped)




FPS samples · Voronoi membership · Delaunay graph · covering radii · occupancy & radius histograms — all computed on ~20.8k real points.

# The three detectors firing on real data




Coverage gaps (insertion points) · separation violators · vanishing cells — same scene, real LiDAR.

# How much do consecutive frames differ?

- Idea: measure the difference by how much the sampling must ADAPT, frame to frame.
- Method (phase_3/temporal_demo.py):
  - Frame 0 — run FPS to get samples S.
  - Each later frame — carry S forward, run Phase 3 correct() against the new cloud.
  - It patches S (insert at gaps, evict vanished/redundant) via cheap one-hop updates.
- Headline signal: edits requested = insertions + evictions.
  - Static scene → ~0 edits · fast-changing scene → many edits.
- 10 Hz data: once the sampling settles, edits fall to 0–2 per frame.

# Two threshold layers — only one decides patch vs. rebuild

- Detector thresholds — decide HOW MANY edits a frame wants. Data-adaptive:
  - Coverage gap = 2.0 × the frame's own median covering radius.
  - Separation = 0.5 × median sample-to-sample nearest distance.
  - Vanishing = occupancy < 1. (each can be pinned to an absolute value.)
- Budget — decides PATCH vs. FULL REBUILD:
  - If (insertions + evictions) > budget → discard S, re-run FPS from scratch.
  - A fixed integer you choose (default 40) — a compute ceiling, not derived from data.
- Discussion point: the detector bar is relative; the budget is absolute.

# Measuring quality: Chamfer, split into two halves

- Chamfer(P, S) = symmetric mean of nearest-neighbour distances (same as extract/).
- Reported as its two directed halves — they catch different failures:
  - cover (cloud → S): typical cloud point's distance to nearest sample → UNDER-COVERAGE.
  - faith (S → cloud): typical sample's distance to nearest real point → STALE samples sitting in empty space.
- chamfer = cover + faith (metres).
- Fresh FPS has faith ≈ 0 — its samples ARE cloud points.
- Baseline check (--baseline): rebuild fresh FPS at the SAME M; ratio = chamfer / fps_chamfer. ratio > 1 ⇒ reuse worse than rebuild.

# Result: reuse is sound, but ~2× the chamfer of a rebuild


- Edits settle to 0–2/frame; budget never tripped (continuous 10 Hz drive).
- One-hop reuse is exact — matches a full recompute; converges, doesn't diverge.
- But chamfer ratio settles ≈ 2.0:
  - reused S sits ~2× farther from the cloud than a fresh rebuild of equal size.
- Why: the faith (staleness) half grows 0 → ~2 m and dominates the gap.

# Terminal output (KITTI, excerpt)

| frame | pts | M | edits | ins | evt | cover | faith | chamfer | fps_cham | ratio | note |
|---|---|---|---|---|---|---|---|---|---|---|---|
| 0 | 33863 | 64 | - | - | - | 2.514 | 0.001 | 2.515 | - | - | FPS baseline |
| 1 | 32720 | 64 | 0 | 0 | 0 | 2.566 | 0.745 | 3.311 | 2.530 | 1.31 | |
| 5 | 36454 | 63 | 1 | 1 | 0 | 2.915 | 1.659 | 4.575 | 2.503 | 1.83 | |
| 10 | 37721 | 54 | 1 | 0 | 1 | 3.487 | 2.024 | 5.511 | 2.521 | 2.19 | |
| 15 | 39933 | 48 | 0 | 0 | 0 | 3.336 | 1.855 | 5.192 | 2.974 | 1.75 | |
| 20 | 40925 | 47 | 1 | 0 | 1 | 3.268 | 2.171 | 5.439 | 2.416 | 2.25 | |
| 25 | 40031 | 45 | 1 | 1 | 0 | 3.429 | 2.334 | 5.764 | 3.063 | 1.88 | |
| 29 | 49254 | 39 | 0 | 0 | 0 | 3.486 | 1.658 | 5.144 | 2.850 | 1.80 | |

edits collapse to 0–2 once settled · faith (staleness) climbs · ratio holds ≈ 2.0 throughout.

DISCUSSION

# Caveat & open questions

- Relative thresholds can hide coarsening:
  - as evictions outpace insertions, M drifts down (64 → ~40), cells grow,
  - the 'gap' bar rises with them → system reports '≈0 edits' while quietly getting coarser.
- The Chamfer metric is what exposed this — edits alone would not have.

- Candidate fixes (should pull chamfer toward the fresh-FPS baseline):
  - Absolute coverage threshold instead of a per-frame relative one.
  - Constant-M mode: pair each eviction with an insertion / top up with FPS.
- Open: is the ~2× staleness cost acceptable for the compute saved? Where is the crossover?